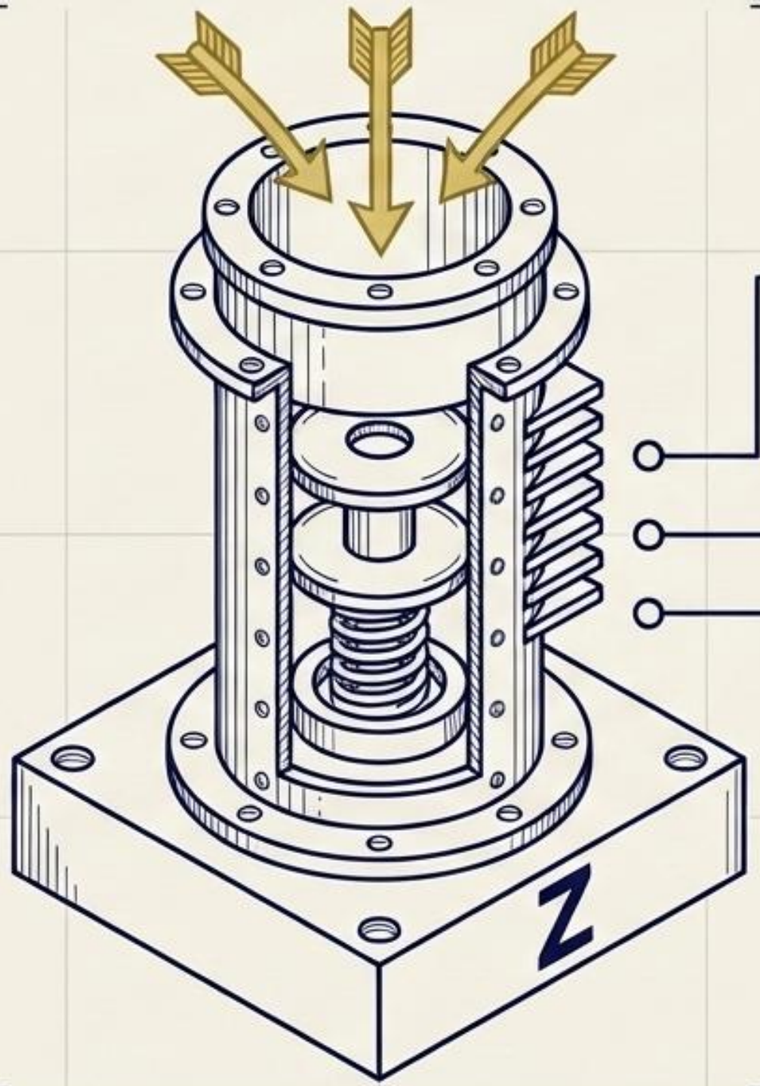
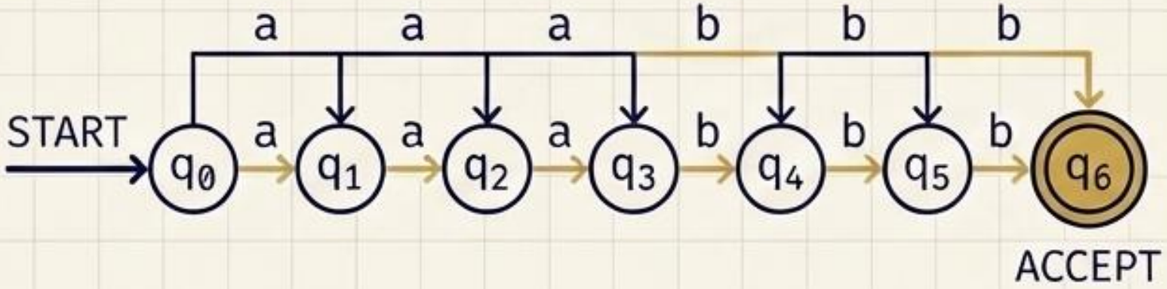


AUTÓMATAS DE PILA: EL PLANO COMPUTACIONAL

NotebookLM

Exitoso



REGEX / Autómata Finito:
Detecta patrones secuenciales.

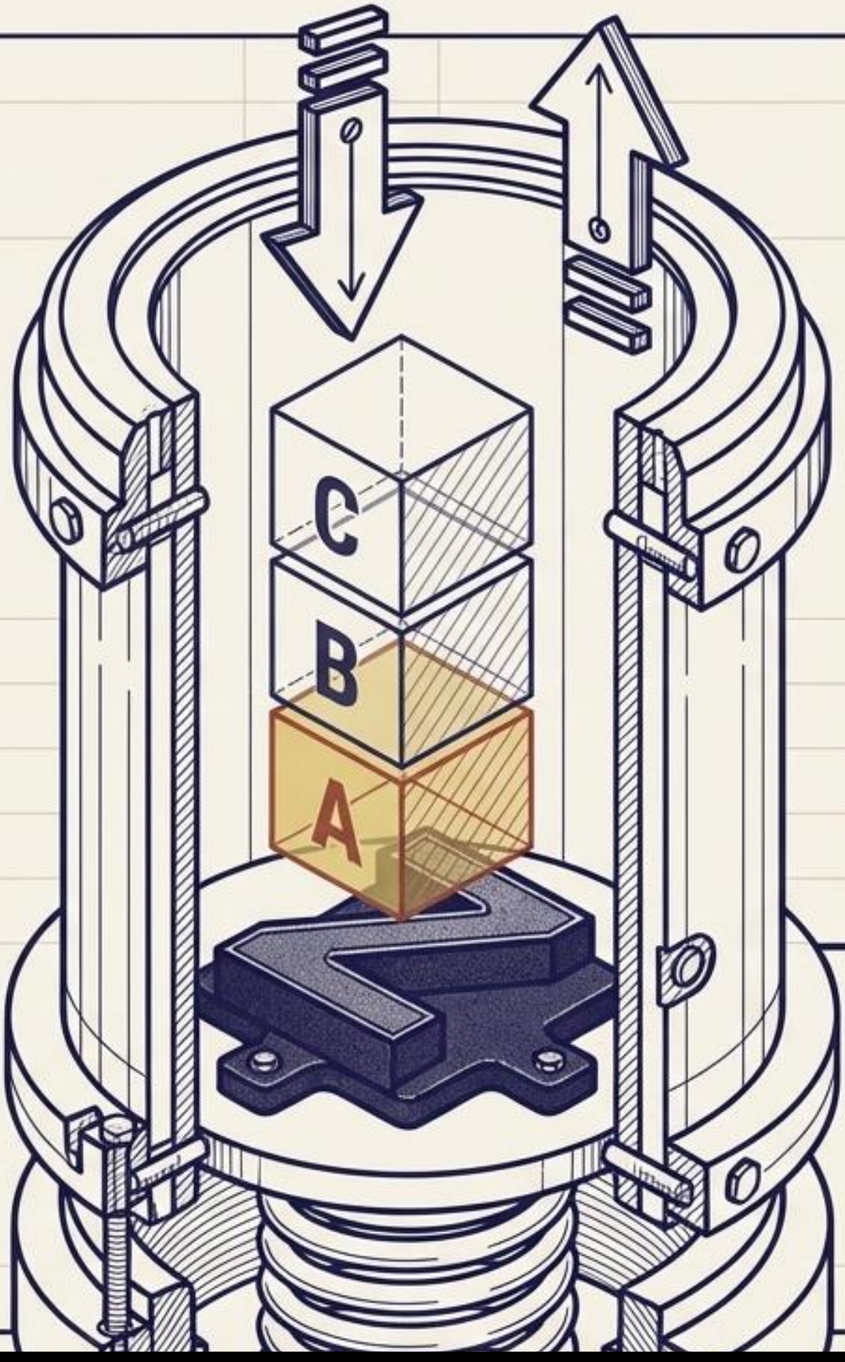
Falló

$a^n b^n$ (con $n>1$)

Lo que no podíamos representar con una REGEX tampoco podemos detectar si una cadena pertenece al leguaje con un AF.

Un Autómata Finito no tiene memoria geométrica. No puede asegurar matemáticamente que entraron la misma cantidad de a que de b.

NotebookLM



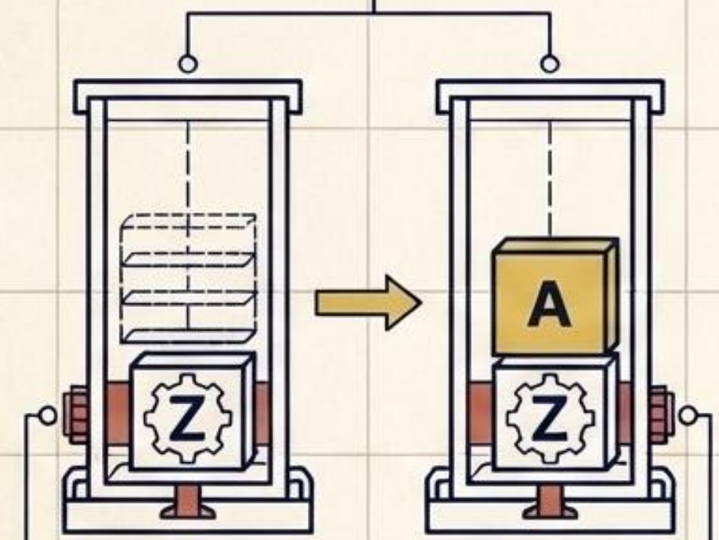
$$AP = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$$

Z: El Fondo de Pila

Si Z no es único y exclusivo, el **autómata no podría diferenciar** entre el "fondo" de su memoria y un **“símbolo apilado”, rompiendo la lógica de aceptación.**

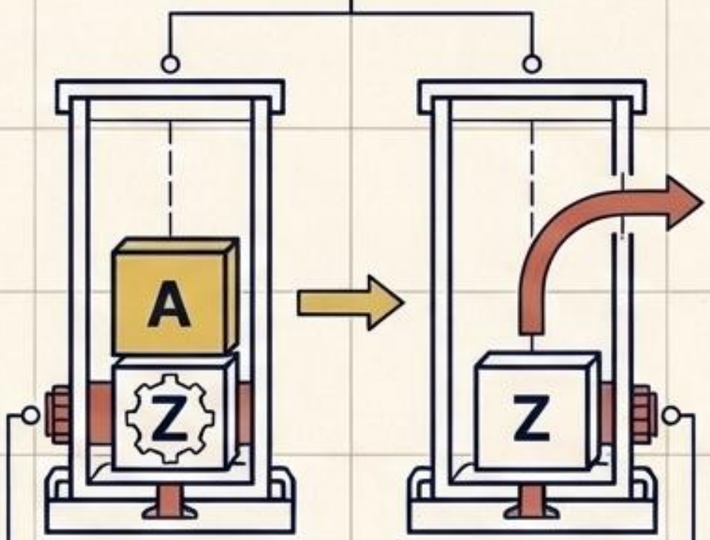
NotebookLM

APILA



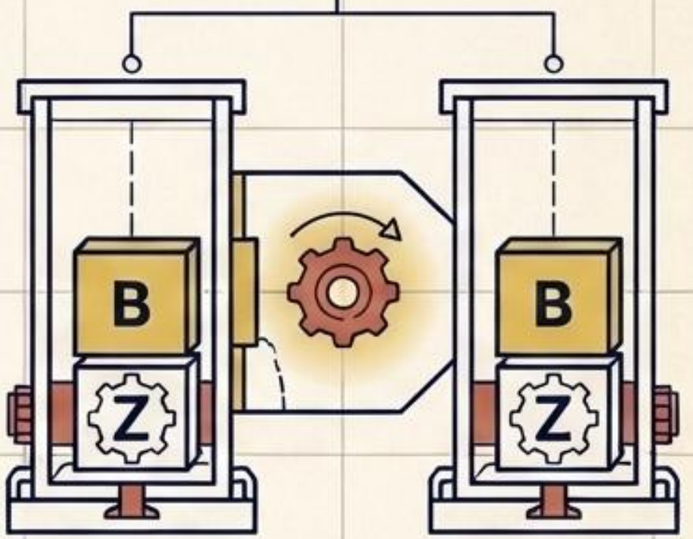
Agrega un símbolo al tope de la memoria.

DESAPILA



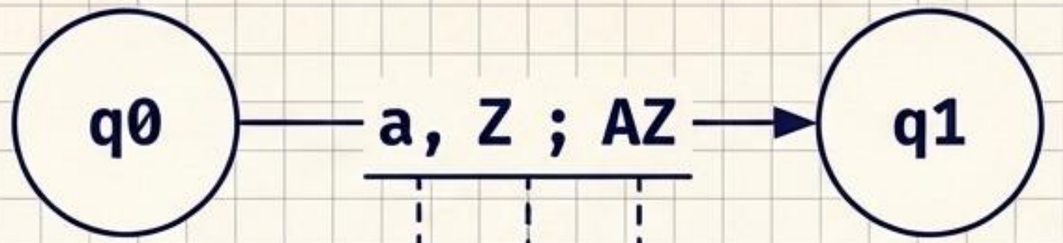
Retira y consume el símbolo del tope.

MANTIENE



Cambia de estado lógico interno sin alterar el contenido físico de la memoria.

NotebookLM



CONDICIÓN DE LECTURA

[LEO: a]

Qué símbolo ingresa por los caracteres de entrada.

CONDICIÓN DE MEMORIA

[TOPE: Z]

Qué debe haber obligatoriamente en la cima de la pila.

ACCIÓN MECÁNICA

[RESULTADO: AZ]

El nuevo estado de la cima (Apilo A sobre Z).

Todo movimiento requiere leer el entorno (caracteres de entrada) y verificar la memoria (tope de pila) antes de actuar.

NotebookLM

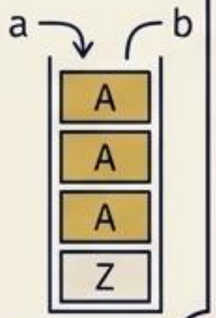
EL ESPEJO DIRECTO

Lenguaje

$a^n\ b^n$

Estrategia

Apila una "A" por cada "a".
Desapila por cada "b".
Termina en Z.



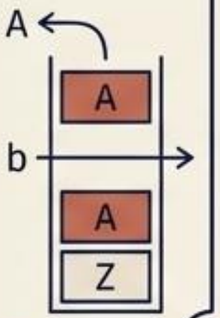
EL PUENTE IGNORADO

Lenguaje

$a^n\ b^m\ c^n$

Estrategia

Apila con "a".
Lee "b" sin modificar la pila.
Desapila con "c".



EL MULTI-EMPAREJAMIENTO SECUENCIAL

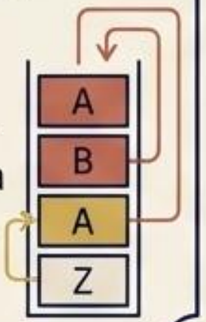
Lenguaje

$a^n\ b^m\ c^m\ d^n$

Estrategia

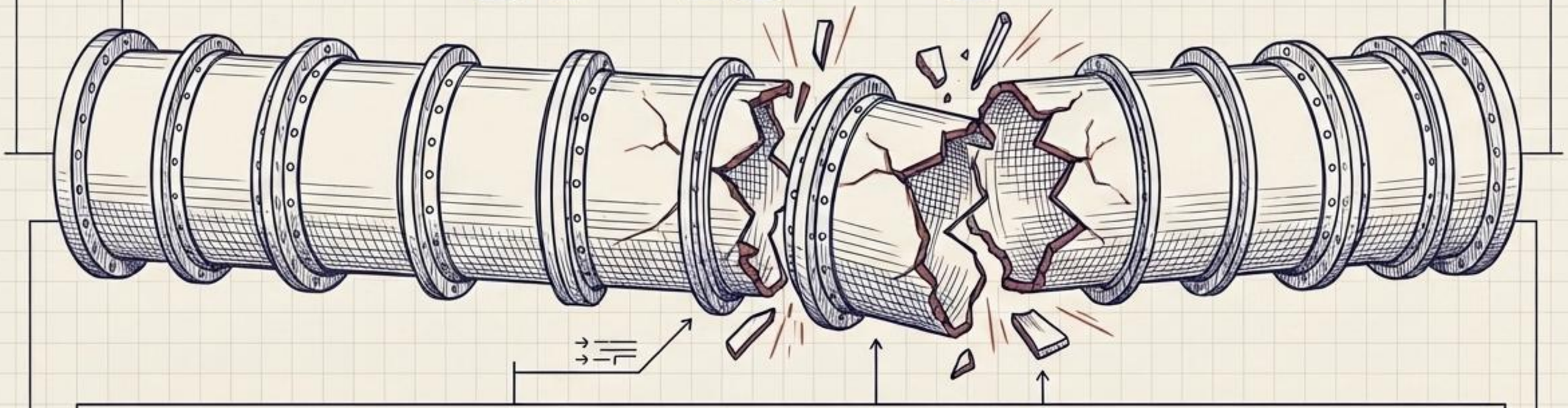
Apila "A" por "a".
Apila "B" por "b".
Desapila "B" por "c".
Desapila "A" por "d".

Funciona porque el orden de anidación (m dentro de n) respeta la arquitectura LIFO. LIFO.



NotebookLM

$a^n b^n c^n$

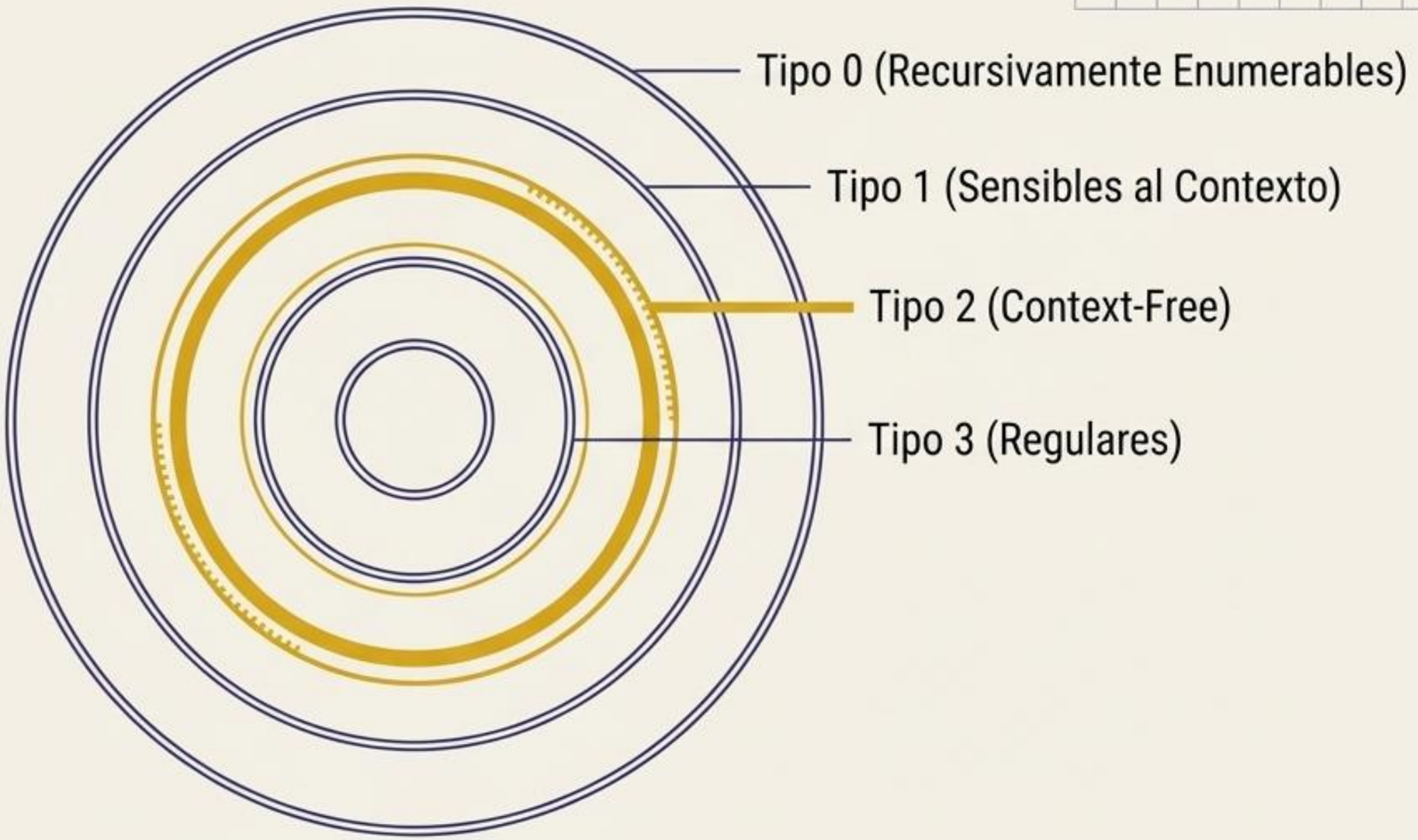


1. Apilamos "A" para contar a^n. (La pila crece).
2. Desapilamos para verificar b^n. (La pila se vacía hasta **Z**).
3. Llega c^n ... pero la memoria de **n** se ha destruido irrevocablemente.

Una pila LIFO solo puede emparejar dos patrones entrelazados. Un tercer requerimiento rompe la geometría de la memoria.

NotebookLM

- Las expresiones que exigen cruzar dependencias múltiples (como contar a, b y c de forma interdependiente) exceden la arquitectura de una sola pila.
- Estas cadenas no representan un Lenguaje Independiente del Contexto (LIC).



El Autómata de Pila nos enseña que el límite de la computación Tipo 2 no está en el tamaño de la memoria, sino en la topología temporal de cómo esa memoria puede ser escrita y borrada.

Fin

NotebookLM